

# Analysis of a Memetic Algorithm on Boolean Satisfiability SAT

Obreja Rares

December 15, 2025

## Abstract

Boolean Satisfiability (SAT) is a fundamental NP-complete problem with numerous applications in verification, planning, and artificial intelligence. In this paper I tried to solve 10 SAT instances from small ones to very large ones (up to 22500 literals) using a Memetic Algorithm. The algorithm used starts with a population of candidate solutions and through multiple selection processes across many generations it gets to better and better solutions. I also used the local search of Hill Climb Best Improvement Algorithm in order to fine tune individual solutions and get closer to the optimum (solving the SAT instance). In the end I managed to get really close to solving all the instances (within  $\approx 1\%$  of the clauses satisfied on the largest ones) but I could completely satisfy only the small instances used while on the medium ones I got within less than 0.5%. Because I didn't have a lot of time to run the instances, I ended up using a small population and few generations on the largest instances which gave worse results than I would have gotten with both a larger population and more generations.

## 1 Introduction

The Boolean Satisfiability problem (SAT) asks whether there exists an assignment of truth values to Boolean variables such that a given propositional formula evaluates to true. SAT was the first problem proven to be NP-complete, and it plays a central role in computational complexity theory.

Genetic Algorithms are population-based stochastic optimization techniques inspired by natural evolution. They use a population of solutions and through crossover and mutation selects future generations in hope of finding the optimum solution within the population through multiple generations. The selection process can be optimized by modifying the way we calculate the fitness function for each candidate therefore, we can have a more relaxed constraint so that slightly worse solutions also have a good change of being selected, or we can be very rigid and only allow the best ones in the next generation which results in an elitism approach. Another important aspect is the number of crossovers and mutations we allow which can give a huge exploration within the population or making it remain basically unchanged after

these processes. The Genetic Algorithms have been successfully applied to many combinatorial optimization problems, including SAT. A Memetic Algorithm is a variant of the Genetic Algorithm that also uses the local search of a Hill Climbing Algorithm in order to fine tune individual solutions.

The purpose of this paper is to analyze the effectiveness of a Memetic Algorithm for solving 10 SAT instances from small ones toward really large ones (up to 22500 literals).

## 2 SAT Problem Definition

A SAT instance is typically represented in *Conjunctive Normal Form* (CNF), defined as a conjunction of clauses, where each clause is a disjunction of literals. A literal is either a boolean variable or its negation.

Formally, a CNF formula is given by:

$$F = C_1 \wedge C_2 \wedge \dots \wedge C_m,$$

where each clause:

$$C_i = (l_{i1} \vee l_{i2} \vee \dots \vee l_{ik}).$$

The goal is to find an assignment to the variables such that all clauses are satisfied.

## 3 The Memetic Algorithm used and its implementation

A Memetic Algorithm is a variant of a Genetic Algorithm that also uses a Hill Climbing Algorithm in order to fine tune the individual solutions. For the SAT problem I also processed the initial data in order to reduce the complexity of the given problem. The algorithm implemented generates a random first population based on the pattern given after processing the input data. If in a clause there was only one literal then I set its value from the start which can't be later modified by the Hill Climbing Algorithm or by mutation. The same happens for the literals that only appear in one form (simple or negated). Then I apply Hill Climb Best Improvement on the initial population in order to improve it a bit before the selection processes. For the selection process I used the wheel of fortune method with a permissive fitness function (when trying a more strict one the population ended up after a few generations with a lot of copies of just one individual). After selecting the next population I applied crossover randomly on some of them with a small fine tune after with the Hill Climb Algorithm. After the crossover I used mutation with a small probability to allow some changes within the individuals used. After all the generations I took the five best solutions and applied again Hill Climb Best Improvement and returned the best one of them.

### 3.1 Implementation

- Solutions - represented as a vector or bits with the length being the number of literals ( $sol[i] = 0$  if the literal  $i$  is false and  $sol[i] = 1$  if the literal evaluates to true)
- Input processing:
  - if the clause has only one literal  $i \rightarrow$  update pattern[i]
  - if a literal appears only negated or not  $\rightarrow$  update pattern[i]
- pattern[i]  $\rightarrow$  0/1 for the literals that must have that value  
 $\rightarrow$  -1 for the rest of the literals
- Population Size - 50 (large instances) / 100 (medium and small instances)
- Number of Generations - 100 (large instances) / 200 (medium and small instances)
- Crossover probability - 0.4
- Mutation probability -  $1/nr\_bits$  (nr of literals)
- Fitness function -  $1.0/(f(x_i) - selection\_pressure + eps)$
- Selection pressure (in order to give better chances at being selected toward the better candidates) -  $worst\_of\_gen$
- eps (used to control the selection pressure) -  $3 * (best\_of\_gen - worst\_of\_gen)$   
(I tried using a smaller eps in order to have a stricter selection but after a few generations the population converged to only one individual)
- step\_nr (the number of steps in the Hill Climb Best Improvement Algorithm in order to improve the individual solution a bit but not wasting time improving to the end) - 100

## 4 3-SAT instances used [2]

Small:

- 1. 50 var - 218 clauses (Uniform Random-3-SAT)
- 2. 100 var - 430 clauses (Uniform Random-3-SAT)
- 3. 150 var - 645 clauses (Uniform Random-3-SAT)

Medium:

- 4. 200 var - 1065 clauses (Uniform Random-3-SAT)

- 5. 600 var - 2550 clauses (Large Random 3-SAT instances)
- 6. 1000 var - 4250 clauses (Large Random 3-SAT instances)
- 7. 2000 var - 8500 clauses (Large Random 3-SAT instances)

Large:

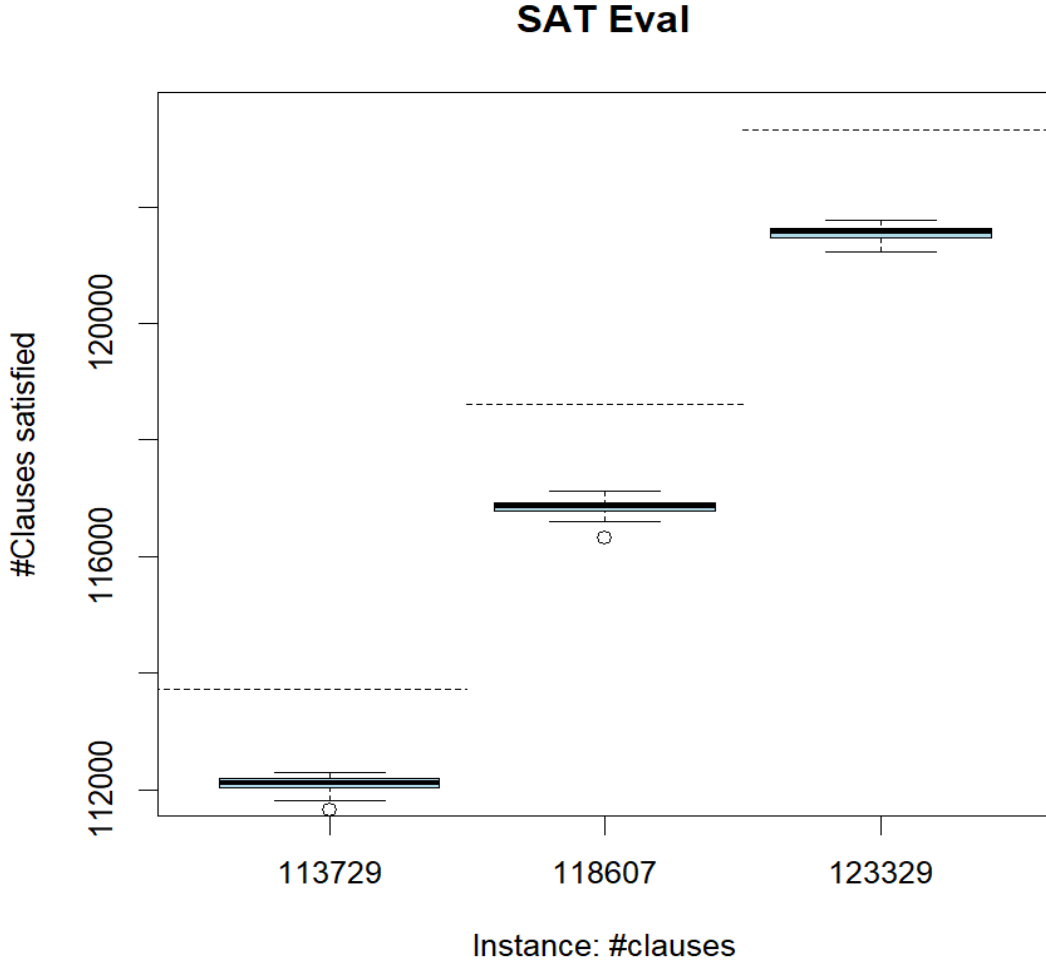
- 8. 21000 var - 113729 clauses (Benchmark instances of SAT Competition Beijing)
- 9. 21800 var - 118607 clauses (Benchmark instances of SAT Competition Beijing)
- 10. 22500 var - 123329 clauses (Benchmark instances of SAT Competition Beijing)

## 5 Experimental Results

I ran the algorithm on each instance 30 times in order to be able to draw a robust conclusion on the results. Due to time constraints the large instances were run with a population size of 50 and 100 generations, while for the rest of the instances I used a population size of 100 and 200 generations.

| Instance(#lit/#cl) | MAX    | %Opt          | MIN    | MEAN       | StdDev  | AvgRunTime(ms) |
|--------------------|--------|---------------|--------|------------|---------|----------------|
| 50 / 218           | 218    | <b>100</b>    | 217    | 217.889    | 0.319   | 154            |
| 100 / 430          | 428    | <b>100</b>    | 430    | 428.778    | 0.485   | 1551           |
| 150 / 645          | 642    | <b>100</b>    | 645    | 644.083    | 0.649   | 2376           |
| 200 / 1065         | 1061   | <b>99.906</b> | 1064   | 1062.75    | 0.732   | 8229           |
| 600 / 2550         | 2534   | <b>99.765</b> | 2544   | 2540.111   | 2.214   | 51816          |
| 1000 / 4250        | 4234   | <b>99.624</b> | 4223   | 4228.5     | 3.112   | 126397         |
| 2000 / 8500        | 8431   | <b>99.529</b> | 8460   | 8450.972   | 5.794   | 404119         |
| 21000 / 113729     | 111641 | <b>98.73</b>  | 112285 | 112086.861 | 145.802 | 2985267        |
| 21800 / 118607     | 116327 | <b>98.754</b> | 117129 | 116849     | 139.324 | 3109640        |
| 22500 / 123329     | 121239 | <b>98.744</b> | 121780 | 121537.111 | 143.136 | 3195345        |

Table 1: Results of the Memetic Algorithm on the used instances



## 6 Conclusions

In this paper, we investigated the effectiveness of a Memetic Algorithm for solving Boolean Satisfiability (SAT) instances of increasing size and difficulty. The proposed approach combines a Genetic Algorithm with a Hill Climb Best Improvement local search and incorporates problem-specific preprocessing to reduce the search space. This combination allowed the algorithm to efficiently explore the solution space while also exploiting local improvements.

The experimental results show that the algorithm is capable of completely solving all small SAT instances and achieving near-optimal solutions on medium and large instances. For the medium-sized problems, the algorithm consistently satisfied more than 99.5% of the clauses, while for the largest benchmark instances, containing over 21,000 variables and more than 110,000 clauses, it still achieved around 98.7% clause satisfaction. These results demonstrate the robustness of the memetic approach even under strict time and resource constraints.

The use of Hill Climb Best Improvement proved to be particularly beneficial

in improving individual solutions both at initialization and after crossover, leading to faster convergence and more stable results. At the same time, a permissive fitness function was necessary to maintain population diversity and avoid premature convergence, especially on larger instances. The preprocessing step also contributed to performance improvements by fixing forced literals and reducing the effective problem size.

Despite these positive results, the algorithm was limited by the small population size and number of generations used for large instances due to runtime constraints. It is reasonable to expect that using larger populations and allowing more generations would further improve solution quality and potentially lead to fully satisfying assignments even for the largest instances.

## References

- [1] Eugen Croitoru, *Genetic Algorithms*, Site: <https://profs.info.uaic.ro/eugen.croitoru/teaching/ga/>.
- [2] *SATLIB - Benchmark Problems*, Site:<https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html>.
- [3] *Boolean satisfiability problem*, Site:[https://en.wikipedia.org/wiki/Boolean\\_satisfiability\\_problem](https://en.wikipedia.org/wiki/Boolean_satisfiability_problem).
- [4] Peter Maandag, *Solving 3-SAT, July 2, 2012*, Site:[https://www.cs.ru.nl/bachelors-theses/2012/Peter\\_Maandag\\_\\_\\_3047121\\_\\_\\_Solving\\_3-Sat.pdf](https://www.cs.ru.nl/bachelors-theses/2012/Peter_Maandag___3047121___Solving_3-Sat.pdf).
- [5] Sid Mijnders, Boris de Wilde, and Marijn J. H. Heule, *Symbiosis of Search and Heuristics for Random 3-SAT*, Site:[https://www.cs.utexas.edu/~marijn/publications/ALDS\\_LaSh.pdf](https://www.cs.utexas.edu/~marijn/publications/ALDS_LaSh.pdf).
- [6] *Hill Climbing in Artificial Intelligence*, Site:<https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/>.
- [7] Chinnnasamy, Sathiyaraj and Ramachandran, Amudha, Ramu, Kurinjimalar, *A review on hill climbing optimization methodology*, Recent Trends in Management and Commerce, vol.3, 2022.
- [8] Selman, Bart and Gomes, Carla P, *Hill-climbing search*, Encyclopedia of cognitive science, vol.81, 2006.
- [9] Pablo Moscato, Carlos Cotta *A Gentle Introduction to Memetic Algorithms(2003)*. Site:[https://www.researchgate.net/publication/313051410\\_A\\_Gentle\\_Introduction\\_to\\_Memetic\\_Algorithms](https://www.researchgate.net/publication/313051410_A_Gentle_Introduction_to_Memetic_Algorithms).
- [10] Abraham D. Flaxman, Microsoft Research *Algorithms for Random 3-SAT*. Site:<https://www.math.cmu.edu/~adf/research/rand-sat-algs.pdf>.